

RESEARCH ARTICLE OPEN ACCESS

AI-Augmented Real-Time Collaborative Blended Modeling Framework for Automotive Embedded Systems

Misbah Mehboob Awan¹  | Muhammad Waseem Anwar^{1,2}  | Wasi Haider Butt¹  | Farooque Azam¹ 

¹Department of Computer and Software Engineering, College of Electrical and Mechanical Engineering, National University of Sciences and Technology (NUST), Islamabad, Pakistan | ²Division of Computer Science and Software Engineering, Mälardalen University, Västerås, Sweden

Correspondence: Misbah Mehboob Awan (mmehboob.cse19ceme@ce.ceme.edu.pk)

Received: 13 September 2025 | **Revised:** 14 January 2026 | **Accepted:** 26 January 2026

Guest Editor: Nadeem Sarwar

ABSTRACT

Model-driven engineering (MDE) has emerged as a foundational paradigm for the development of embedded systems in domains such as automotive, where stringent correctness and structural consistency must be maintained throughout the engineering process. However, traditional modeling practices often suffer from fragmented toolchains, siloed notations, and limited support for real-time collaboration across distributed stakeholders. This paper presents an artificial intelligence (AI)-augmented and role-aware collaborative framework that enables blended modeling through seamless synchronization of textual and tree-based/graphical views, underpinned by a robust service-oriented architecture. Rooted in automotive system engineering standards, specifically electronics architecture and software technology-architecture description language (EAST-ADL), and instantiated via an industrial-grade domain-specific modeling language (DSML) use case, the proposed framework facilitates stakeholder-specific modeling via both textual and tree-based/graphical editors. At the core of this infrastructure lies a real-time coordination engine that governs session orchestration, conflict resolution, and model merging, ensuring semantic consistency across heterogeneous representations. A key innovation is the integration of an explainable AI (XAI) based log analysis engine, which leverages large language models (LLMs) to generate human-readable summaries of user activity, model evolution, and design rationale. This not only enhances transparency and auditability but also introduces explainability as a first-class concern in collaborative MDE workflows. Empirical validation confirms the framework's effectiveness in enabling synchronized, traceable, and fault-tolerant co-modeling for automotive embedded systems. By fusing collaborative computing, XAI, and domain-specific rigor, this research advances the state of the art in intelligent, scalable, and human-centered system modeling environments.

1 | Introduction

The growing complexity of embedded systems, particularly in the automotive domain, demands modeling approaches that ensure real-time performance, reliability, and traceability. Development typically involves distributed teams including engineers, domain specialists, verification experts, and software developers working across multiple abstraction levels and notations. The central challenge lies in enabling efficient collaboration while maintaining

model coherence, transparency, and validation across heterogeneous environments.

Model-driven engineering (MDE) has become a standard approach in automotive development, offering abstraction and early analysis, yet conventional toolchains remain monolithic, with limited support for real-time multiuser collaboration, role-specific perspectives, and dynamic evolution tracking. These shortcomings are amplified when textual and tree/graphical representations must be maintained in parallel. Blended modeling

This is an open access article under the terms of the [Creative Commons Attribution](https://creativecommons.org/licenses/by/4.0/) License, which permits use, distribution and reproduction in any medium, provided the original work is properly cited.

Copyright © 2026 Misbah Mehboob Awan et al. *IET Software* published by John Wiley & Sons Ltd.

addresses this by combining textual precision with graphical intuitiveness, but existing solutions lack robust synchronization, fault recovery, conflict resolution, and explainability.

Domain-specific modeling languages (DSMLs) extend MDE by tailoring abstractions to domain needs. Electronics architecture and software technology–architecture description language (EAST–ADL), for instance, supports layered semantics for automotive functions, deployments, and timing, but lacks distributed collaboration, traceability, and automated reconciliation across textual and structural views. Blended modeling across web and desktop platforms further introduces fragmented toolchains, versioning inconsistencies, and limited interpretability of collaborative contributions. A cohesive framework is, therefore, required to unify EAST–ADL–inspired textual models with graphical/tree structures while enabling real-time collaboration, fault-tolerance, and explainable traceability. Motivated by these challenges, this research is guided by two central questions:

RQ1. How can a real-time collaborative blended modeling framework be systematically designed to maintain semantic consistency, role-aware conflict resolution, and synchronized multiview representations across heterogeneous textual and graphical notations in automotive embedded system development?

RQ2. How can explainable artificial intelligence (XAI) techniques be integrated into collaborative blended modeling environments to enhance traceability, interpretability, and awareness of model evolution, while preserving determinism, privacy, and modeling correctness?

To address these research questions, we propose an AI–augmented real-time collaborative blended modeling framework tailored to automotive embedded systems. The framework harmonizes textual (EATEXT: EAST–ADL Textual) representation and graphical/tree-based (EAXML: EAST–ADL XML) representation modeling in a unified environment, reinforced by role-based access control (RBAC), secure editing workflows, and centralized synchronization.

A key innovation is its AI–driven explainable logging, which records user actions with metadata and generates natural language summaries via LLMs, enhancing transparency and managerial oversight. Complementing this, a runtime persistence and backup mechanism archives intermediate states at fixed intervals, ensuring resilience against failures and recovery of work-in-progress.

As illustrated in Figure 1, the architecture supports distributed collaboration through real-time synchronization, automated conflict handling, and role-aware branching/merging. Users contribute via web-based textual editors or desktop graphical interfaces, while session tracking, automated checkpoints, and metadata-annotated merges maintain semantic alignment and accountability. This hybrid environment integrates XAI with fault-tolerant persistence to deliver a secure, scalable, and transparent modeling solution for automotive embedded systems. This work makes the following key contributions:

- First, it presents an AI–augmented real-time collaborative blended modeling framework that unifies EAST–ADL–inspired textual (EATEXT) and tree/graphical (EAXML) representations within a semantically coherent, role-aware environment. The framework formalizes collaboration semantics through explicit merge and conflict-handling rules and a real-time model-merging protocol that ensures controlled integration, provenance preservation, and structural consistency across distributed modeling sessions. It further incorporates a fault-tolerant persistence layer enabling periodic checkpointing, automatic recovery, and temporally ordered versioning, and introduces an XAI log analysis engine that utilizes on-device large language model (LLM) inference to generate privacy-preserving and human-readable summaries of model evolution.
- Second, it delivers a technically grounded implementation architecture that spans DSML grammar selection, blended multiview synchronization, metadata-enriched merge management, and heterogeneous web/desktop interaction. The design demonstrates how synchronization protocols, conflict

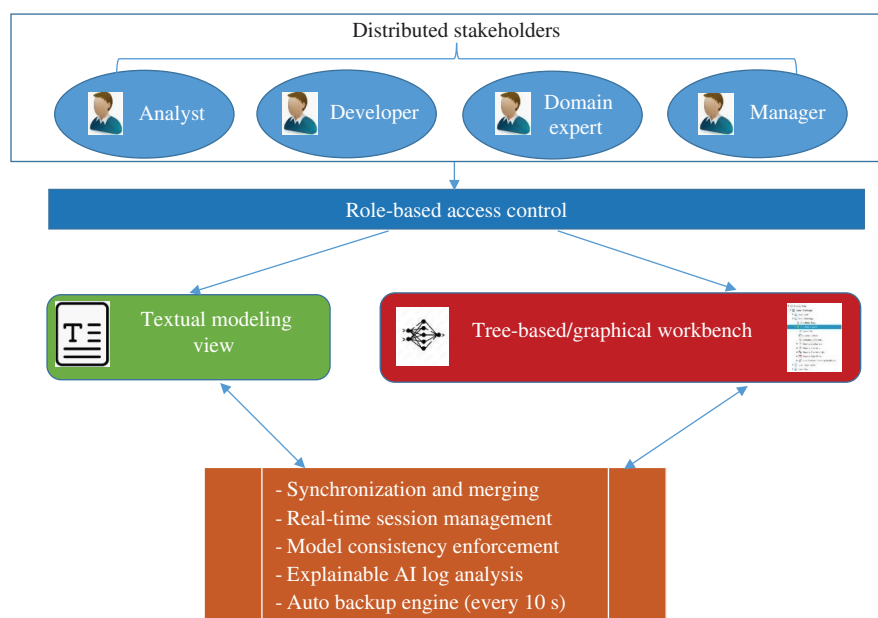


FIGURE 1 | Overview of role-based multiview blended modeling environment with real-time collaboration.

semantics, and XAI-driven traceability are operationalized in a practical and reproducible toolchain.

- Third, it provides a complete proof-of-concept realization using Xtext-based textual modeling, Eclipse Modeling Framework (EMF)/EAST-ADL tool platform (EATOP) structural rendering, and microservice-based role, merge, and persistence services. The framework is validated through a car wiper case study from Volvo and evaluated via performance benchmarking with up to 100 concurrent users, demonstrating operational feasibility, scalability, and architectural robustness.

Together, these contributions establish a rigorous, extensible, and experimentally grounded foundation for collaborative modeling in automotive embedded system development.

The remainder of this paper is structured as follows: Section 2 reviews related work in collaborative MDE and blended modeling. Section 3 presents the proposed framework, its architecture, and interaction mechanisms. Section 4 outlines the implementation, including tooling and deployment across web and desktop platforms. Section 5 reports the proof-of-concept evaluation on an automotive case study, while Section 6 discusses performance results. Section 7 critically analyzes implications and limitations, and Section 8 concludes with future research directions.

2 | Literature Review

MDE is widely used to address the complexity of embedded system design and verification by enabling platform-independent modeling and early validation. DSMLs further enhance this process by providing tailored abstractions for specific domains. However, effective collaboration in DSML-based development remains challenging, particularly when geographically distributed teams must coordinate across heterogeneous textual and tree/graphical model representations.

2.1 | Collaborative Model-Driven Software Engineering

Collaborative model-driven software engineering enables distributed stakeholders to codevelop, refine, and analyze models by combining principles of MDE with computer-supported cooperative work [1]. Tools such as WebGME [2], Eclipse EMF/GMF [3], and AToMPM [4] provide model sharing and concurrent editing, yet most lack fine-grained role management, real-time synchronization, or seamless integration of textual and graphical views. Key challenges remain in merging versions, resolving semantic conflicts, and ensuring traceability. While several merging techniques exist, they are rarely embedded into scalable modeling environments that support versioning and visualization for engineers, verifiers, and managers.

WebGME [2] offers online collaboration, but limited conflict resolution. AToMPM [4] supports multiple concrete syntaxes and real-time updates, but relies on simplistic conflict handling. MONDO [5] enables offline/real-time collaboration via version control, though its dependency on language-specific UIs limits extensibility. Collaboro [6] provides collaborative support for DSL-based modeling, but struggles with scalability, while

Parsafix [7] bridges MPS and eclipse, but lacks extensibility. FlexiSketch [8] introduces mobile sketch-based modeling, yet is limited in scalability and concurrency. In contrast, our framework integrates extensible hybrid editors, real-time synchronization, role-based contributions, and AI-enhanced traceability for enterprise-grade collaboration.

2.2 | Collaborative Blended Modeling Environments

The landscape of blended modeling has evolved significantly, with recent frameworks specifically addressing the rigorous demands of safety-critical embedded systems [9]. Collaborative blended modeling environments integrate textual and graphical/tree-based DSML views, aligning tools with diverse stakeholder preferences. Van Tendeloo and Vangheluwe [10] propose separating versioning of abstract and concrete syntax, but without detailing synchronization strategies. Exelmans et al. [11] add optimistic versioning with semantic merges, yet omit persistent storage and role-based workflows. A layered collaboration framework [12] improves traceability and artifact management, but remains tool-centric, lacking provisions for real-time conflict handling or explainability.

Commercial and open-source reviews by David et al. [13] highlight underdeveloped collaboration features in blended modeling. Tools like Umple [14], based on SVN-like merging [15], handle syntax, but miss semantic conflict resolution. BUMBLE studies [13, 16, 17] explore blended modeling prototypes, including synchronous collaboration [17], but without robust versioning, conflict detection, or inconsistency management. Our framework extends these efforts with domain-agnostic synchronization, extensible hybrid editors, and AI-assisted summarization for collaborative transparency.

2.3 | Automotive Systems and Collaborative Blended Modeling Environments

Automotive embedded systems introduce unique constraints of safety, timing, and traceability that generic collaborative platforms often overlook. EAST-ADL [18] defines abstractions for automotive architectures and supports both textual and graphical representations through its meta-model; however, existing modeling environments that implement EAST-ADL typically operate in single-user settings and do not provide real-time collaborative editing or role-based conflict handling. Our framework addresses this limitation in current EAST-ADL-based modeling environments by introducing distributed and role-aware collaborative editing enriched with XAI activity logs, integrated persistence, and coordinated conflict resolution. These capabilities enable multiuser development for automotive architectural models while fully preserving EAST-ADL semantics and ensuring consistent and traceable evolution of system artifacts.

2.4 | Gaps and Research Motivation

Current frameworks lack unified hybrid modeling support, integrated conflict resolution, persistent storage, or explainable tracking of contributions; limitations that hinder distributed DSML development in domains where system models must remain consistent, traceable, and rigorously managed, such as automotive embedded

systems. To our knowledge, no solution offers real-time collaboration tailored to automotive DSMLs with EAST-ADL semantics, synchronized multiview editing, and explainable collaboration traceability. To address this, we propose an AI-augmented hybrid collaborative blended modeling framework that unifies textual and tree/graphical modeling across web and desktop platforms, enables real-time multiuser interaction, and embeds robust mechanisms for conflict resolution, persistent versioning, and AI-driven log summarization. Positioned at the intersection of collaborative modeling, blended engineering workflows, and XAI, the framework offers a scalable and domain-relevant solution for distributed model-driven development in automotive embedded systems.

3 | Proposed Framework

We propose an AI-augmented real-time collaborative blended modeling framework for automotive embedded systems. It unifies automotive DSMLs, role-aware collaboration, blended modeling, and XAI to ensure semantic integrity, traceability, and accountability in distributed development.

As shown in Figure 2, the architecture blends textual and tree/graphical views with real-time synchronization, conflict resolution,

and model merging. While motivated by modeling needs in safety-critical automotive systems, the framework itself is not a safety-critical component; rather, it supports developers working in such domains by ensuring domain conformance, artifact traceability, and temporal consistency across distributed modeling activities. Section 3 outlines the architecture, its core components, and the operational workflow underlying these capabilities.

3.1 | Architecture Overview

The proposed framework enables real-time, role-aware, and semantically synchronized collaborative modeling for automotive embedded systems, combining textual and tree/graphical views with modularity, traceability, and safety compliance. At its core, the collaborative modeling and processing engine orchestrates authentication, sessions, versioning, concurrency, and validation, while linking with the XAI engine and persistence layer for transparency. Access is governed by RBAC aligned with ISO 26262 [19], mapping roles to permissions. Heterogeneous clients, including web-based Xtext DSML editors and desktop EATOP-style editors, remain synchronized via a custom protocol using *.eaxml*. Embedded domain constraints and explainable logging ensure reproducibility, accountability, and resilience.

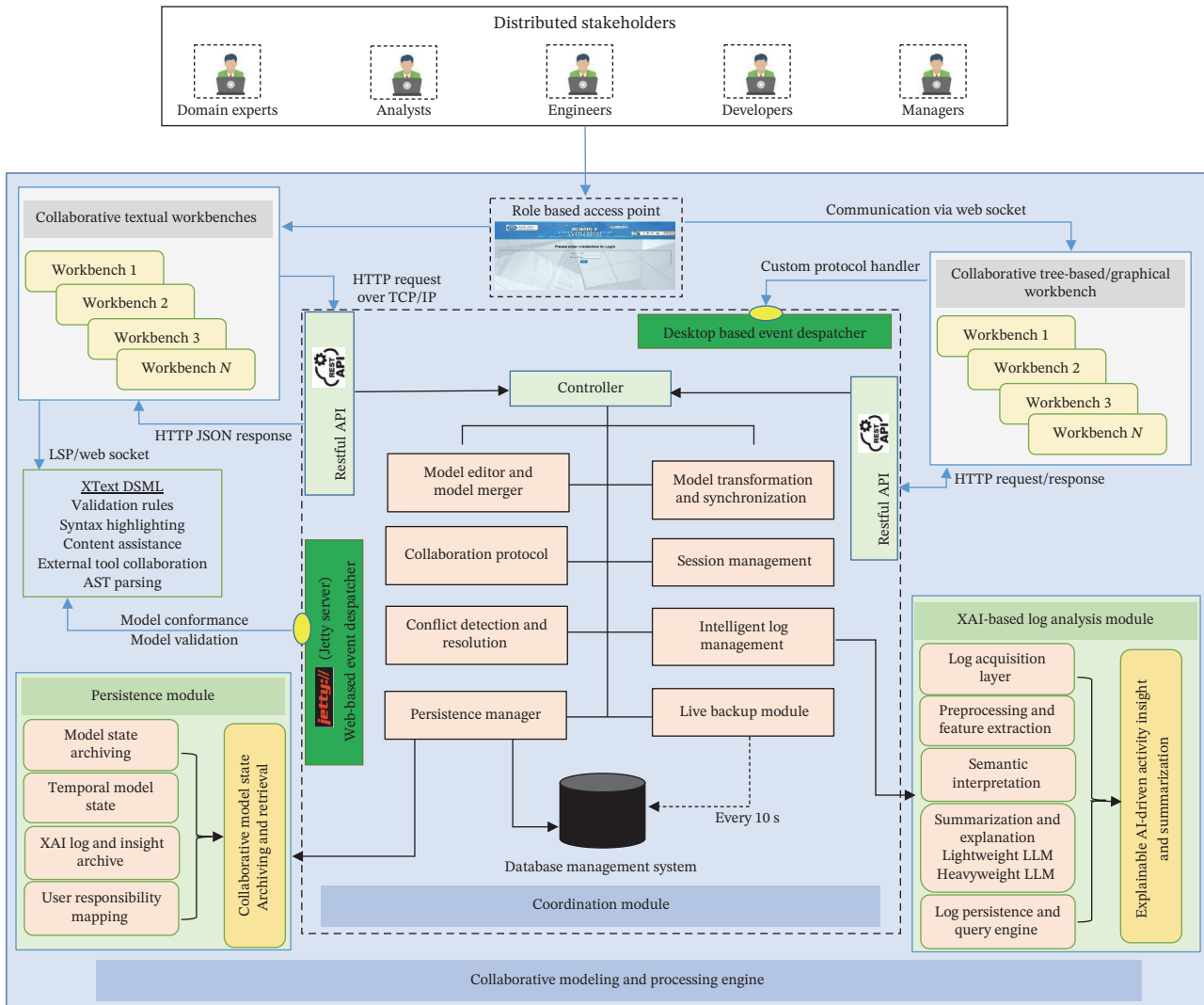


FIGURE 2 | Real-time collaborative blended modeling framework for embedded systems with explainable artificial intelligence.

3.2 | Blended Multiview Modeling Support

The framework provides a structured mechanism for blended multiview modeling, addressing a widely acknowledged need in automotive systems engineering, where stakeholders interact with models through different notations, abstraction levels, and cognitive preferences. The approach keeps textual and tree-based/graphical representations semantically aligned, synchronized, and editable in real time. The textual view is supported by an Xtext-based grammar that serves as the formal backbone by providing automotive rules, compliant constructs, embedded validation rules, and syntax-aware editing while supporting integration with external verification tools. The tree-based/graphical view, derived from *.eaxml* serialization, offers a visual abstraction that supports architecture inspection and traceability navigation. While empirical usability studies remain part of future work, the unified semantic model and continuous bidirectional synchronization between views are intended to support blended modeling practices and maintain correctness and traceability across representations within automotive development workflows.

3.3 | Collaborative Access via Role-Based Entry Point

Collaboration begins through a semantically aware RBAC layer that governs user entry and tool provisioning. Upon authentication, stakeholders are mapped to formal roles (e.g., *SystemArchitect*, *TimingAnalyst*, and *SafetyEngineer*) and redirected to role-specific modeling environments via a custom protocol handler that enforces access policies. The routing mechanism adheres to the following rule-based formalization:

```
<RoleAssignment> ::= IF Auth(User) AND UserRole
    ∈ Roles THEN Redirect(User, Platform
    [UserRole])
```

Web-based textual modeling is supported through an EAST-ADL aligned DSL editor offering validation, semantic highlighting, AST parsing, and interoperability with external verification pipelines. In parallel, desktop-based tree/graphical modeling uses *.eaxml* representations to provide hierarchical and graphical views, with bidirectional synchronization ensuring semantic equivalence:

```
<SyncRule> ::= Δ (Model_T) ↔ Δ (Model_G) iffSemantics
    (Δ) ∈ Preserved
```

This rule ensures every delta (Δ) between textual (*Model_T*) and tree-based/graphical (*Model_G*) views maintains semantic coherence before state propagation.

Cross-platform edits are encapsulated into lightweight delta events:

Delta packet structure:

```
Δ_Event := (UserID, Role, OperationType, ModelFragment, Timestamp)
```

These packets are routed through the collaborative engine, which enforces synchronization, integrity, and traceability. The RBAC subsystem regulates privileges dynamically:

```
<PermissionRule> ::= PERMIT (Operation, ModelScope)
    ⇐ Role ∈ AuthorizedRoles
```

3.4 | Collaborative Modeling and Processing Engine

The collaborative modeling and processing engine is the framework's core middleware, coordinating user actions, role constraints, and synchronization protocols to ensure that concurrent edits produce syntactically valid and semantically consistent models. It manages sessions and state life cycles, dispatches event-driven deltas enriched with user, role, and time stamp metadata, and performs real-time transformations between textual and graphical/tree views. Conflicts are resolved through rule-based strategies such as precedence ordering and overlap prevention, while a version-controlled persistence layer enables differential backups, snapshots, and merged state recovery. Complemented by explainable logging for traceability and an event dispatcher for synchronous and asynchronous communication, it leverages in-memory caching and distributed synchronization to minimize latency and maintain coherent views. Together, these features provide high-availability and conflict-resilient collaboration consistent with the safety requirements of automotive embedded systems.

3.5 | Model Editor and Merger

At the user interaction layer, the model editor and semantic merger provides a role-aware and constraint-sensitive environment that enables both individual authoring and collaborative integration of automotive embedded models. Unlike conventional syntactic diff utilities, it performs semantic merging guided by EAST-ADL rules and model-aware heuristics. User interactions are captured as delta events, each annotated with:

- Operation type (create, modify, and delete).
- User metadata (role, ID, and session ID).
- Time stamp.
- Semantic footprint (affected AST node, constraint category, and cross-references).

These deltas are processed by a rule-based conflict resolver employing policies such as:

- First valid contributor wins for orthogonal edits.
- Time stamp priority rule for overlapping changes.
- Constraint violation blocking to prevent domain rule breaches (e.g., duplicate START values).

The merger engine ensures the unified model:

- Preserves automotive DSL meta-model conformance.
- Embeds contributor metadata for traceability.
- Passes structural and semantic validation postmerge.
- Remains ready for visualization, transformation, or verification.

This semantically aware and runtime-adaptive pipeline ensures conflict-resilient, accountable, and domain-compliant collaboration in automotive modeling.

3.5.1 | Model Merging Strategy in Collaborative Environment

A central feature of the framework is its ability to generate a consolidated model view that integrates contributions from all participants, supporting shared understanding in role-based collaborative modeling. The process begins when a reviewer or manager requests the merged view, prompting the system to retrieve the latest runtime instances via data access objects (DAOs) services; unlike simple concatenation, a multilayered merging protocol is applied to preserve semantics, maintain contributor traceability, and ensure coherence, anchored on the super user's runtime instance, which serves as the reference backbone containing the validated grammar constructs and top-level structure. This base model is enriched with metadata (such as authorship, time stamps, and validation results) before sequentially incorporating syntactically and semantically verified fragments submitted by other contributors. Each integrated fragment retains lightweight metadata for traceability. Conflicts are resolved through either first-valid-contributor-wins or time stamp-priority policies, while lightweight annotations embed provenance details in the final model. The collaboration protocol synchronizes this merged structure across textual and graphical editors, ensuring distributed MDE workflows remain syntactically correct, semantically consistent, and traceable. By grounding merging rules in meta-model constraints (e.g., EAST-ADL), the framework provides robust, real-time integration for real-time collaborative modeling, as stated below:

Collaborative model merging rules

1. MR-1: Reference model rule. A validated reference model M_{ref} is established as the baseline for all subsequent integration steps to ensure structural stability.
2. MR-2: Role-scope rule. Each contribution C_i is accepted only if its edits lie within the contributor's authorized scope S_i ; edits, where $C_i \not\subseteq S_i$ are excluded.
3. MR-3: Validity rule. A contribution is eligible for integration only when it satisfies structural and semantic checks, expressed as $V(C_i) = \text{true}$.
4. MR-4: Conflict resolution rule. When multiple contributions target the same model element, the selected update is determined using a precedence function P that prefers higher-authority roles or, when applicable, more recent edits.
5. MR-5: Provenance rule. Each incorporated edit is assigned a provenance tag p_i capturing contributor identity, time stamp, and action metadata to maintain traceability.
6. MR-6: Consistency rule. After processing all contributions, the merged model M_{merged} is synchronized across all modeling views to ensure a unified and consistent state.
7. MR-7: Offline reintegration rule. Offline edits are restored only to the user's own session upon reconnection and published as a provisional and noncommittable revision. They

are merged into the authoritative model exclusively after manual approval by a higher-role reviewer, who validates semantic correctness and compatibility with concurrent online changes.

Technical significance: The merging protocol 1:

1. Preserves meta-model fidelity across heterogeneous editing environments.
2. Encapsulates role-aware governance for distributed modeling teams.
3. Ensures deterministic merge outcomes under concurrent edit conditions.
4. Facilitates explainable postmerge analysis through embedded provenance.

This rule-based abstraction enables the framework to remain domain-agnostic while supporting the modeling needs of regulated environments such as automotive, aerospace, and industrial embedded systems, where traceability, structural rigor, and consistent artifact evolution are essential.

The final merged model instance is then serialized and delivered to the requesting user via `HttpResponse`, with full support for both browser-based display and tree-based/graphical rendering on the desktop side. This ensures that all users, whether working in textual or tree-based/graphical editors, are presented with a unified and comprehensible snapshot of the collaborative modeling effort.

This structured model merging not only supports traceable integration of contributions but also lays the groundwork for XAI-based summarization, which further interprets and narrates the evolution of the model overtime based on contributor logs and semantic deltas.

Algorithm 1 formalizes the merge cycle executed whenever distributed contributions are integrated into the collaborative model. The process begins by anchoring all operations on the authoritative reference model M_{ref} , ensuring a stable and validated baseline. Each incoming contribution C_i is then examined in a staged manner: its acceptance is conditioned on role-based authorization, offline-originated edits require high-role approval before being eligible for integration, and syntactically or semantically invalid contributions are isolated in a quarantine state. For valid contributions, each model element e is inspected for structural or semantic conflicts with the current reference model; conflicts are resolved according to predefined precedence rules before the element is incorporated. Metadata capturing contributor identity and time stamps is embedded to preserve provenance.

The final phase generates a unified merged model M_{merged} by applying all accepted modifications to the reference state. The line “Serialize $M_{\text{merged}} \leftarrow M_{\text{ref}}$ ” indicates the transformation of the updated in-memory abstract model into its persistent representation (e.g., `.eatxt` or `.eaxml`), ensuring that the merged state is stored in a consistent and platform-independent format. Serialization guarantees that all clients-web and desktop-can load the same canonical version of the model. Once serialized, the merged instance is broadcast to all collaborators so that every view remains synchronized with the authoritative state.

1. Initialize Merge

$M_{\text{ref}} \leftarrow$ primary reference model

2. Process Contributions

For each contribution C_i :

If user not authorized for C_i 's scope: reject and continue

If C_i originates from offline editing and lacks high-role approval: mark as provisional and continue

If C_i fails validation: quarantine and continue

For each element e in C_i :

If e conflicts with M_{ref} : resolve conflict

Embed metadata and integrate e into M_{ref}

3. Finalize and Distribute

Generate M_{merged} from the updated M_{ref}

Serialize $M_{\text{merged}} \leftarrow M_{\text{ref}}$

Broadcast M_{merged}

The collaborative model merging protocol (Algorithm 1) applies the defined rules (MR1–MR6) to unify distributed model updates into a semantically consistent representation. It begins by establishing a stable reference model (MR1), filters contributions according to authorized scope (MR2), and validates accepted fragments using syntactic and semantic checks (MR3). Overlapping edits are handled through policy-based conflict resolution grounded in role precedence and temporal ordering (MR4). Integrated elements are annotated with provenance metadata for traceability (MR5), after which the final merged model is serialized and propagated to maintain system-wide consistency (MR6). Together, these steps provide a deterministic and role-aware foundation for collaborative embedded system modeling.

To accommodate heterogeneous usage scenarios, the framework supports offline editing within the desktop environment and reintegrates these changes through a controlled and role-aware process. When operating offline, a user's edits are stored locally and restored into that user's personal session upon reconnection, without directly modifying the shared authoritative model. The resulting revision is visible to other collaborators as a provisional instance, enabling awareness without risking unintended overwrites. Final incorporation into the global model occurs only after a manual review by a higher-privileged role, such as an architect or designated reviewer, who verifies semantic consistency, dependency ordering, and potential interactions with concurrently submitted online edits. This staged reintegration strategy ensures that delayed updates do not compromise model stability and that users can clearly perceive and validate any changes arising from offline work.

3.6 | Collaboration Protocol for Real-Time Editing

A core contribution of the framework is the collaboration protocol, which ensures semantically consistent and temporally synchronized coediting across heterogeneous clients in distributed

automotive development. It aligns textual (EAText) and structural/tree-based (EAXML) views by combining WebSocket channels for low-latency bidirectional updates with HTTP RESTful services for reliable transactional operations such as saves, merges, and conflict resolutions. This hybrid design balances immediacy with robustness, while its platform-neutral and model-agnostic nature enables interoperability across DSMLs like EAST–ADL. Functioning as a scalable synchronization fabric, the protocol preserves model correctness, reduces overhead from concurrent edits, and fosters a unified shared model, thereby establishing a rigorous foundation for real-time and role-aware collaborative modeling.

3.7 | Fault Tolerance and Model Integrity Assurance (Live Backup Module)

Ensuring robustness and data integrity is important in collaborative modeling workflows used within safety-regulated domains such as automotive embedded systems. To support this need, the proposed framework incorporates a fault-tolerant integrity assurance module, implemented as an autonomous and time-triggered checkpointing mechanism that archives the modeling session every 10 s without user intervention.

Each checkpoint provides a consistent serialization of the model state, enabling recovery with minimal deviation from the last stable version in the event of hardware failures, power outages, or client-side termination. Functioning as a temporal redundancy layer, this mechanism aligns with classical principles of forward error recovery and fail-safe persistence, ensuring continuous operation without rollback to distant prior states. Critically, it addresses nonfunctional requirements mandated by safety standards such as ISO 26262 and IEC61508, preserving model integrity under adverse conditions including hardware faults, network disruptions, or process failures.

3.7.1 | Operational Principle

The fault tolerance mechanism is a time-triggered, transaction-safe checkpointing service operating at a fixed cycle period:

$$T_c = 10 \text{ s.}$$

At each cycle, the current authoritative model state M_t is serialized to persistent storage as an atomic snapshot. This mechanism ensures that, in the event of a disruption, recovery can deterministically resume from the most recent committed state without rollback to stale versions.

Fault tolerance preservation rules:

FT-R1 (deterministic periodicity): Checkpoints occur at a fixed T_c regardless of editing activity, ensuring predictable fault-coverage intervals.

FT-R2 (atomicity): Each checkpoint operation is all-or-nothing; partially written states are discarded to prevent inconsistent recovery.

FT-R3 (concurrency isolation): Checkpoint processes run in dedicated execution threads, isolated from user transaction threads to prevent lock contention.

FT-R4 (semantic synchronization): In multiuser scenarios, checkpoints capture only semantically validated global model instances to prevent propagation of transient inconsistencies.

FT-R5 (deterministic recovery): Restoration always resumes from the latest valid checkpoint M_t , where $t \in T_c$, guaranteeing a reproducible recovery trajectory.

FT-R6 (compliance logging): All checkpoint and restoration events are logged with author, time stamp, and checksum for safety audits.

Furthermore, this strategy contributes to continuous model availability and data survivability and key attributes for compliance in automotive safety standards (e.g., ISO 26262). By capturing temporally coherent model snapshots, the module enables postfailure recovery, session restoration, and seamless reintegration into ongoing collaborative workflows.

In essence, the fault-tolerant integrity assurance module not only minimizes the impact of runtime faults but also instills a robust safety net, preserving the continuity and trustworthiness of the model development process under real-world constraints.

3.8 | Persistence Module: Temporal Consistency and Auditability in Collaborative Modeling

In collaborative MDE, domains such as automotive embedded systems, preserving modeling states, activity histories, and model evolution trails are essential for traceability, accountability, and recoverability. To address these needs, the framework incorporates a rigorously engineered persistence module, serving as the temporal backbone for capturing, versioning, and managing artifacts across distributed environments. Beyond basic fault tolerance, the module integrates principles of time-aware repositories, provenance tracking, and role-sensitive activity mapping, thereby supporting both operational reliability and long-term analytical insight.

The module's architecture comprises four subcomponents.

- The model state archiving layer serializes and stores model snapshots at defined intervals or trigger points (e.g., semantic validations or save events), enabling deterministic rollback and reproducible experimentation.
- The temporal event logging layer records every model change with precise time stamps, ensuring reconstruction of model evolution for debugging and compliance audits.
- The explainable log insight archival subcomponent integrates with the AI module to store both raw logs and narrative summaries, semantically indexed for later querying.
- Finally, the user-responsibility map links modeling elements and activities to individual users under role-based access policies, supporting accountability and access auditing.

By embedding these layers, the persistence module moves beyond conventional backup functionality, institutionalizing versioning semantics, lineage tracking, and end-to-end transparency. It enables longitudinal analysis of model evolution, root-cause investigation of inconsistencies, and retrospective evaluation of team performance. In doing so, it provides an operational memory layer that ensures reproducibility and supports structured safety assessments, making collaborative automotive system development auditable, traceable, and compliant with the rigorous demands of standards such as ISO 26262.

3.9 | XAI-Based Log Analysis

The framework integrates an XAI-based log analysis engine to enhance accountability and traceability in collaborative MDE. Each modeling action is encoded as a (*user_id*, *timestamp*, *action_type*, and *affected_element*) tuple and clustered into higher-level episodes such as constraint updates or branch merges. Using NLP and LLMs, the engine transforms raw logs into concise and human-readable narratives, contextualizing technical changes for developers and managers. This approach ensures that collaborative activities are transparent, interpretable, and aligned with automotive embedded systems requirements. The novelty of this approach lies in its multilayered abstraction of modeling actions, semantic enrichment of event streams, and human-readable translation of technical edits into managerial insights. By ensuring that collaboration is not only logged but also explained and contextualized, the framework supports transparent model evolution, auditability, and informed stakeholder communication. This XAI-driven log analysis mechanism, thus, bridges the gap between technical traceability and human interpretability, offering a scientifically grounded solution for trustworthy collaborative modeling in automotive systems.

3.9.1 | Rules for the XAI Log Analysis Engine

Rule 1: Contribution traceability rule: If a user adds or modifies a model element, then log the action with user identity, time stamp, and model context. Formal representation:

```
IF action_type IN {ADD, MODIFY} THEN log_entry :=
{ user_id, timestamp, model_element_id, model_
package, action_type, model_context }
```

Usage: This rule ensures all individual contributions are recorded for traceability.

Rule 2: Conflict detection rule: If two or more users modify the same model element within a short time window (Δt),

Then flag as a potential conflict event and classify the collaboration episode as “concurrent conflict.”

Formal representation:

```
IF (user_A, user_B) :
    modify(model_element) AND t < threshold THEN
    classify_event := "Conflict"
```

Usage: Supports clustering collaborative behaviors for later summarization.

Rule 3: Repetitive correction rule: If the same user modifies the same model element $\geq N$ times within a session,

Then classify as “repetitive correction” and recommend review.

Formal representation:

```
IF count(modify_event(user, element)) > N THEN
    label := "Needs Review",
    suggest_annotation := true
```

Usage: Informs managerial dashboards and summary explanation generation.

Rule 4: Semantic role impact rule: If a user from a high-privilege role (e.g., architect and integrator) modifies an element that others also modified,

Then prioritize that version in the summarization and mark it as a “finalizing action.”

Formal representation:

```
IF user.role {Architect, Integrator} AND modifies
(model_element)
THEN model_element.status := "Finalized"
```

Usage: Helps generate summary statements like “Architect finalized the BrakeSystem configuration.”

Rule 5: Summary generation rule: If a collaboration episode includes $\geq M$ unique users modifying elements within the same package,

Then generate a team-level summary narrative.

Formal representation:

```
IF package :
    count(distinct users modifying elements) >=
M THEN
    summary := "TeamCollaborativelyUpdated
[pkg_name]"
```

Usage: Direct input for LLM summarization prompts or templates.

Rule 6: Timing violation risk detection: Integration with EAST-ADL domain: To specialize these rules for automotive embedded systems (as per EAST-ADL), you can bind model elements like:

·Function, TimingConstraint, AnalysisFunctionType, HardwareComponent into the rules, e.g.:

IF

```
    modify_event involves ExecutionTimeConstraint
AND (start_time, end_time) > max_threshold THEN
    log_risk := true, label := "Timing Violation Risk"
```

3.9.2 | XAI Activity Summarization Pipeline

The framework incorporates a locally deployed LLM to generate natural-language summaries of model evolution, enabling explainable collaboration in real time. The NLP module that we implemented follows a structured, four-stage pipeline as shown in Figure 3 and described as follows:

Stage 1: Service and model availability monitoring: Before any inference request is issued, the system probes the local Ollama runtime using a lightweight HTTP health check. This ensures that summarization is only attempted when the inference environment is available and responsive.

Stage 2: Dynamic model selection: The system automatically identifies a functioning LLM from a predefined list of local models. It performs a short verification query and selects the first model capable of responding successfully. This mechanism enables the environment to adapt to different hardware capabilities without manual configuration.

Stage 3: Prompt normalization and context management: Change information extracted from the modeling logs is embedded into a structured summarization template. To ensure predictable inference times, the system enforces a maximum prompt length; longer inputs are automatically compressed by preserving instructions and selectively truncating less critical model content. This keeps inference responsive during high-activity collaboration sessions.

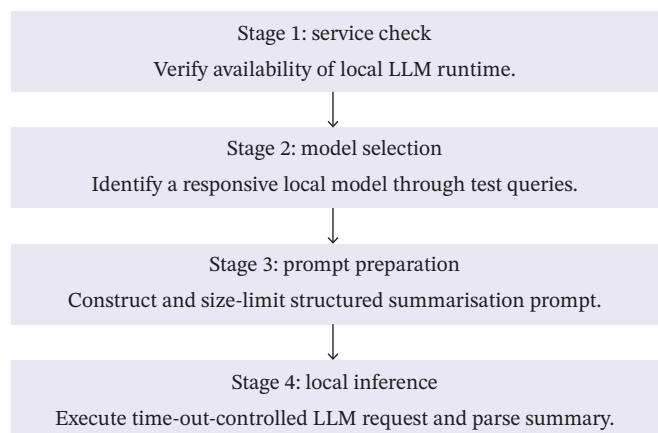


FIGURE 3 | Local XAI activity summarization pipeline implemented in the proposed framework.

Stage 4: Deterministic local LLM inference: Once the prompt is constructed, the system submits a JSON-encoded inference request to the local model. Each request is executed under strict time-out, cancelation, and error-handling conditions to prevent UI blocking. The returned textual response is then parsed, cleaned, and integrated into the activity timeline as a concise human-readable summary.

This pipeline forms an AI-assisted semantic layer that augments collaboration by contextualizing raw model edits into explanations that are easy to interpret.

While the LLM-based summaries enhance interpretability, they are not treated as authoritative analyses. The component operates on validated change logs with constrained and domain-scoped prompts to minimize ambiguity, and all generated narratives remain user-reviewable. This positions the LLM as an assistive explanation layer rather than a correctness-critical element of the workflow.

3.10 | Orchestration and Event Dispatching

The orchestration and event dispatching subsystem ensures synchronized collaboration across heterogeneous platforms by coordinating session states, user roles, and edit sequences. A central coordination module manages a live registry, while web and desktop dispatchers transmit structured events via WebSocket/HTTP or TCP/IP. Using deterministic ordering (Lamport clocks) and role-aware precedence, it guarantees causal and semantic consistency, supports domain constraints (e.g., ISO 26262 checkpointing), and provides a high-assurance layer for conflict-resilient collaboration.

3.11 | Integration and Workflow

The framework unifies textual and graphical modeling through a six-phase workflow: login, role-based interface allocation, edit capture, real-time synchronization and merging, XAI-driven log summarization, and persistence of validated states. As depicted in Figure 2, it integrates editors, orchestration, XAI, and persistence services into a cohesive pipeline. This design ensures fault tolerance, traceability, and semantic correctness, offering a robust foundation for domain-specific collaborative modeling in automotive systems.

4 | Implementation Architecture

The implementation architecture of the proposed collaborative framework has been carefully engineered to support distributed, real-time collaboration in embedded system modeling across both web-based and desktop-based platforms. At its core, the architecture brings together multiple layers, including modeling interfaces, synchronization protocols, transformation engines, persistence mechanisms, and AI-powered analysis to ensure a seamless and intelligent collaboration experience for all stakeholders.

The foundation of the modeling environment is built using the Eclipse Xtext framework, which supports the definition and deployment of domain-specific languages tailored to specific application areas. Each DSML is formally defined using Xtext grammar, which includes syntax rules, semantic constraints, and editor configurations. The generated artifacts from Xtext provide a range of IDE features such as syntax highlighting, code completion, validation rules, and UI support. These features



FIGURE 4 | Runtime instance of the collaborative environment.

are deployed on both web and desktop modeling platforms to ensure consistency in model authoring. Model validation is implemented via Xtext's validator classes, while domain-specific rules can also be integrated via external XML configurations.

Although this grammar layer is necessary for textual editing, it is not part of the collaborative framework itself; the collaboration, synchronization, and merge services operate independently of the chosen DSML. This makes the environment DSML-agnostic, capable of supporting any automotive modeling language such as EAST-ADL, AUTOSAR, or custom extensions. In this work, an EAST-ADL aligned grammar was selected solely as the case study for implementation and validation.

4.1 | Web-Based Collaborative Modeling Environment

The web environment is built on Eclipse Xtext, offering browser-based DSL editors with role-aware access. Upon login, users are routed to personalized model fragments aligned with their responsibilities. The interface, implemented with Eclipse RAP and ACE editor, employs AJAX-based partial rendering for smooth interactions, while a RESTful API manages persistence, merging, and retrieval.

Collaboration is governed by a protocol handling authentication, session tracking, and branching. Each user edits in an isolated DSML editor and simultaneously views a read-only merger editor displaying the consolidated model (Figure 4). To ensure resilience, sessions maintain temporary working copies checkpointed every 10s, complemented by manual persistence. This dual-persistence strategy safeguards against failures such as browser crashes or power loss. The DSML editor provides syntax highlighting, auto-completion, and grammar validation, while the merger view ensures collective awareness. All interactions, including save, merge, and log access, occur via asynchronous AJAX calls, enabling scalable and fault-tolerant multiuser collaboration.

4.2 | Desktop Modeling Environment and Interoperability

The desktop environment integrates textual (.eatxt), tree, and graphical (.eaxml) views of the model, comprising a textual model representation, a hierarchical tree-based view of the model, and a synchronized graphical representation of the model. Edits in one representation are synchronized across others through the framework's transformation engine, ensuring semantic consistency with EAST-ADL constraints. Access is initiated via a custom URI protocol (eatop://...), which launches the desktop client with

user-specific model instances. This supports hybrid workflows where textual precision and graphical intuitiveness are available concurrently, enabling domain experts and engineers to collaborate seamlessly. Updated models are persisted via the back end engine into a shared database, ensuring platform-wide consistency.

4.3 | Model Merger, Synchronization, and Conflict Resolution

The framework's model editor and model merger consolidate collaborator changes at the meta-model level with real-time conflict detection. Overlapping edits trigger notifications and merge options based on predefined rules and user roles, while the collaboration protocol ensures structural and semantic validity, state tracking, and history preservation. The merger engine anchors on the super user's runtime instance, sequentially integrating validated user fragments with contributor metadata. Conflicts are resolved using time stamp-priority or first-valid-entry rules, and the unified model is broadcast across clients. Annotated merges enable traceability and auditing, with AI log analysis providing interpretable summaries and collaborative insights.

4.4 | Back End Collaboration Logic (Model and Controller Layer)

The architecture adheres to the model-view-controller (MVC) pattern, ensuring separation of concerns, maintainability, and scalability. The controller layer is implemented via a custom `ServletController` class, which receives HTTP AJAX requests from the client, delegates processing to the relevant business logic classes, and returns responses to the UI in JSON format.

Each logged-in user is represented via a user-specific session object that maintains credentials, runtime DSML instances, and activity metadata. Sessions are managed using standard `HttpSession` objects. For database interactions, a `UserDAO` class encapsulates all data access logic, including saving runtime instances, managing user sessions, and compiling merged DSML versions.

4.5 | Persistence Engine

All models, along with associated metadata such as user roles, time stamps, and conflict resolutions, are persisted in a central repository backed by MySQL. The persistence engine is responsible for storing both runtime model states and finalized versions after successful merges. It also logs periodic snapshots and records session-specific information to support auditability and rollback if needed. In addition to the model data, the engine stores explainable summaries generated by the AI logging system (discussed next), which further enriches the traceability and documentation aspects of the collaborative workflow.

4.6 | AI-Based Log Analysis Engine

To strengthen explainability within the collaborative environment, we implemented an AI-based log analysis engine that operates as an integral back end service. The engine runs entirely on a local inference layer using the Ollama runtime and supports multiple LLM variants (e.g., LLaMA 3.2 1B/3B, Phi-3Mini, ad

TinyLlama). Upon activation, the engine performs a lightweight service check to ensure the local inference process is reachable and then dynamically selects a functioning model by issuing short verification queries. This dynamic selection mechanism allows the system to adapt automatically to available computational resources and seamlessly fallback to smaller models when heavier ones are unavailable or resource-constrained.

For each collaborative edit, the engine receives a pair of consecutive model snapshots and extracts structural changes using line-based differencing. These changes are then encoded into a structured prompt template that embeds the delta, the responsible user, and the time stamp. A typical prompt constructed by the system takes the form:

```
You are summarizing collaborative modeling
activity.
Changes detected:
Added: state READY
Removed: transition IDLE-> RUN
User: Misbah
Date: 2025-05-14
Time: 12:43:28
Generate a concise human-readable summary.
```

To ensure deterministic behavior, the engine enforces strict limits on prompt length and automatically compresses verbose change logs while preserving all instructional content. The final inference request is executed through a time-out-controlled HTTP pipeline that specifies model name, context window, temperature, and prediction limits. Returned summaries are parsed using a robust JSON extraction routine to eliminate artifacts such as partial completions or malformed tokens.

The generated natural language explanations are stored in the persistence layer alongside traditional logs, enabling team leads and project managers to trace not only what was modified but also the underlying intent inferred from the activity. This creates a transparent and auditable record that connects low-level model diffs with high-level collaborative behavior, thereby extending the framework's accountability and interpretability.

To support reproducibility, the web-based collaborative components of the proposed framework, including synchronization and collaboration services, have been made publicly available on GitHub [20] with setup documentation. The EATOP desktop modeling environment is proprietary and, therefore, cannot be open-sourced. However, the manuscript provides detailed architectural descriptions, formal rules, and case studies to enable conceptual replication and extension using alternative tools.

5 | Proof of Concept—Validation and Evaluation

To validate the effectiveness and practicality of the proposed AI-augmented hybrid collaborative blended modeling framework, we conducted an in-depth proof of concept using a real-world industrial case study. The car-wiper case study from Volvo, provided through the ITEA BUMBLE Project, was selected as the test bed. This industrial-grade example features multiple domain-specific packages and modeling constraints, making it ideal for

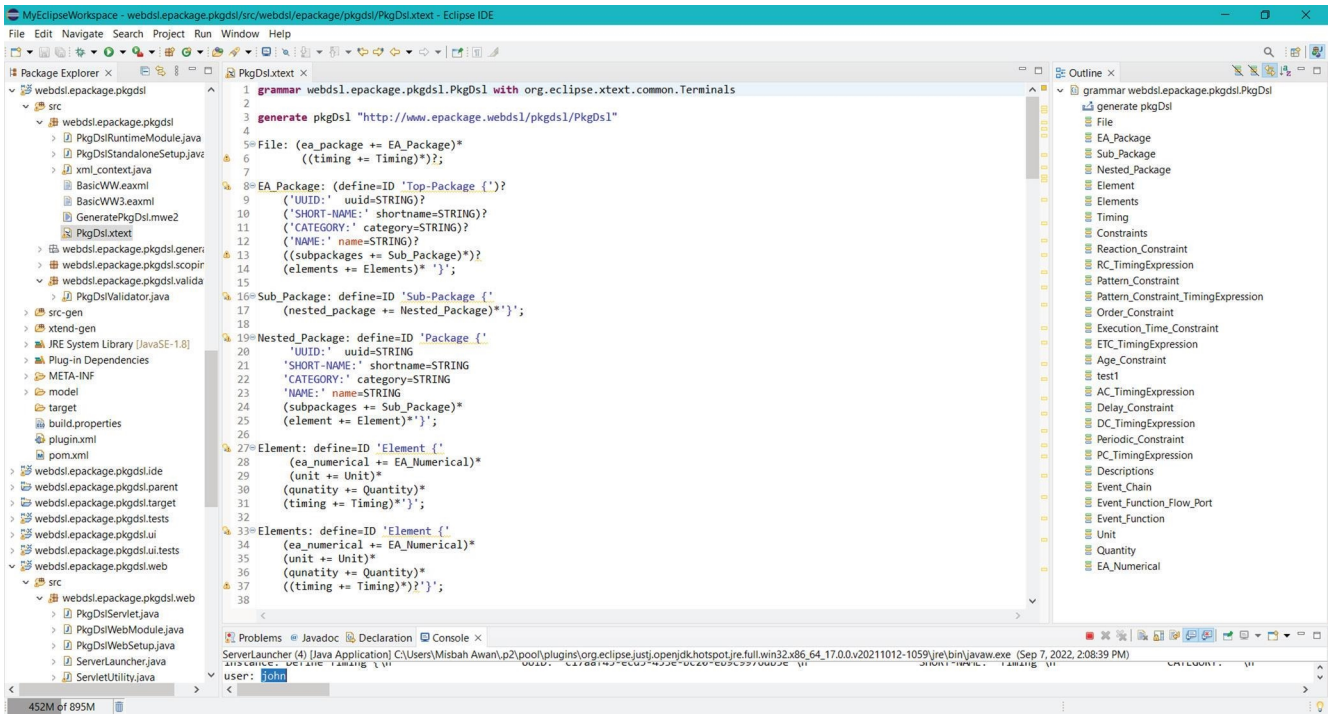


FIGURE 5 | Timing Xtext grammar.

evaluating collaboration, conflict resolution, validation, and explainability mechanisms under real conditions.

5.1 | Collaborative Environment Setup

5.1.1 | DSML Grammar Selection

To validate the framework, we adopted an industrial use case from the ITEA BUMBLE Project, focusing on the Volvo wiper timing package in the automotive domain. This package specifies temporal constraints vital to safety and performance, including periodic constraints (fixed wiper activations), delay constraints (sensor-to-action latency), execution time constraints (motor runtime limits), and reaction constraints (rapid response to heavy rainfall). These constraints were formally modeled in EAXML and transformed seamlessly across textual (EAText), graphical, and tree-based representations within our blended framework. The domain-specific language was implemented using Eclipse Xtext, enabling grammar definition (Figure 5), syntax highlighting, content assistance, validation, and integration. Dedicated testing and deployment support within Xtext further ensured a robust implementation tailored to automotive modeling needs.

5.1.2 | Blended Representations

Figure 6 presents the blended collaborative modeling environment implemented in the proof of concept, illustrating how textual, tree-based, and graphical representations coexist and remain synchronized across both web-based and desktop platforms. The textual editor (Figure 6a) serves as the primary entry point for EAST-ADL-aligned model authoring, while the hierarchical tree view (Figure 6b) exposes the structural decomposition of the model for navigation and inspection. Complementing these, the graphical view (Figure 6c) provides an architectural depiction that supports visual reasoning about functional

interactions and system layout. Edits performed in any of these representations (whether initiated from the web interface or the desktop application) are propagated through the transformation and synchronization engine, ensuring that all views reflect a consistent semantic interpretation of the underlying model. This arrangement enables seamless collaboration across heterogeneous modalities, allowing users to alternate between precise textual manipulation, structural exploration, and visual comprehension without breaking the modeling workflow.

5.2 | Interoperability Between Textual and Tree/Graphical Representations

A key strength of our implementation is its ability to interoperate seamlessly with external modeling artifacts. In this setup, the external EAST-ADL model *BasicWW.eaxml* was imported and integrated into the workflow, enabling cross-tool compatibility and enriching the content-assistance features of our web-based collaborative editor.

In our proposed framework, distinct yet synchronized representations of the model are maintained to support heterogeneous collaboration: the textual model editors are based on the EAText format, while the graphical/tree-based editors rely on the EAXML format. This multirepresentation strategy ensures that both perspectives remain consistent across platforms. When a model is authored or modified via the web-based textual editor, it is initially created in the EAText format. This textual form is then automatically transformed into the corresponding EAXML structure, which is rendered and displayed within the desktop-based graphical/tree editor. Conversely, any changes made in the EAXML-based graphical editor are translated back into EAText format, enabling consistent and up-to-date visualization in the web-based textual interface. This bidirectional transformation pipeline ensures semantic coherence between both views, allowing

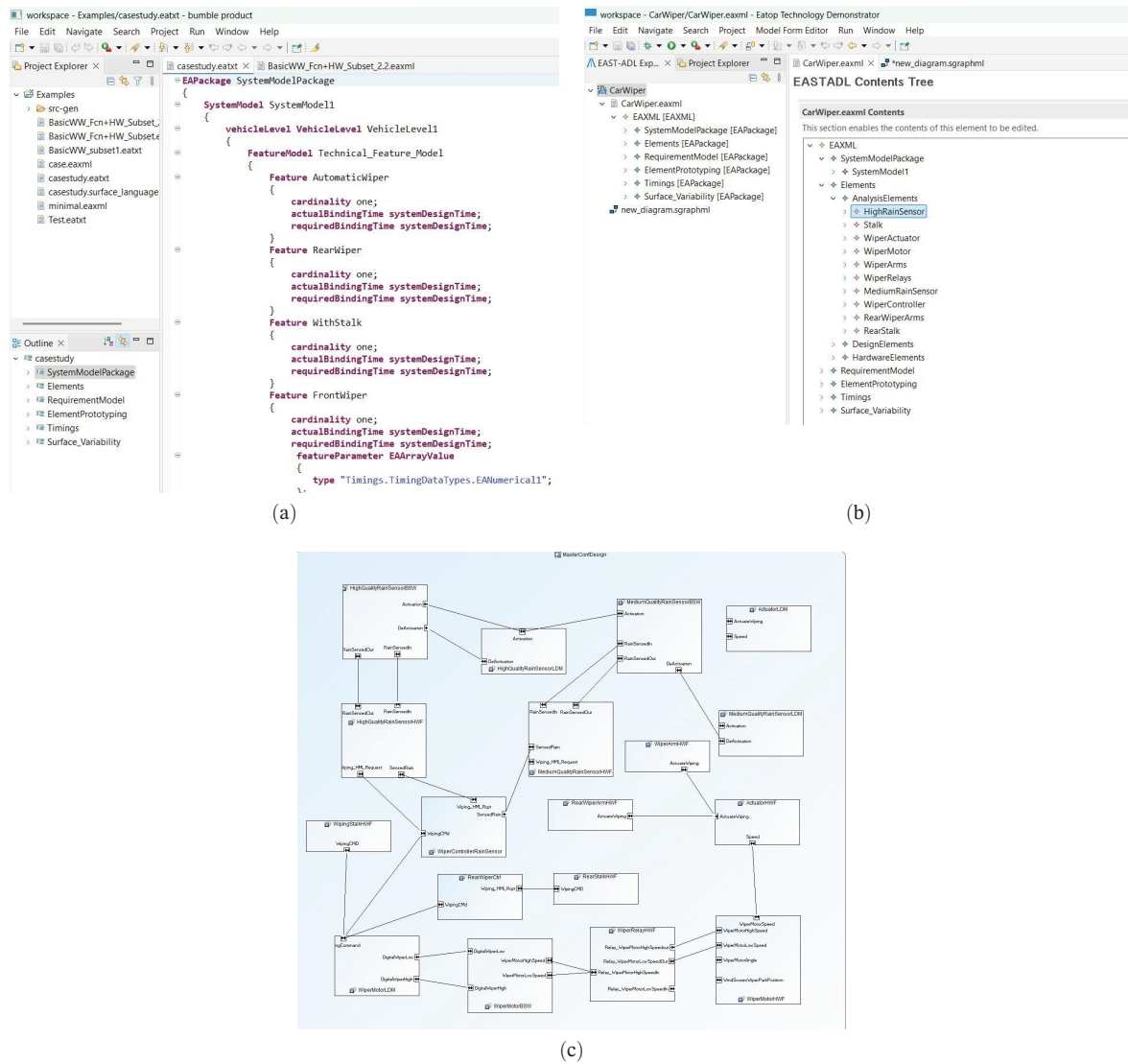


FIGURE 6 | Multiple collaborative modeling environment. (a) Textual model representation. (b) Tree model representation. (c) Graphical model representation.

distributed users to interact seamlessly within their preferred editing environments without compromising model integrity.

5.3 | Validation Rules

To enforce domain-specific correctness and improve early error detection during model development, a set of custom validation rules were defined on the Volvo-specific DSML grammar using the Eclipse Xtext framework. These validation rules serve to ensure semantic integrity and to guide users through real-time feedback during the modeling process.

Rule 1: Error validation for execution time constraints: A critical validation rule was implemented to ensure that within any *Execution_Time_Constraint*, no two *START* elements are assigned the same value. This constraint prevents logical conflicts and timing ambiguities in the execution model. If a violation is detected, the framework immediately highlights the offending construct with a red underline and presents a descriptive error message. This proactive validation mechanism helps maintain model correctness and enforces temporal consistency in the execution semantics.

Rule 2: Warning validation for naming conventions: Another validation rule enforces a naming convention for the *Execution_Time_Constraint* elements. Specifically, it mandates that each constraint name must begin with an uppercase letter. This rule is intended to promote consistency and readability across models developed by distributed teams. Violations of this convention do not halt model execution but are flagged with a yellow warning triangle, encouraging users to correct the issue without interrupting their workflow. Figure 7 illustrates a sample snippet demonstrating this rule, with its practical implications discussed in the following Section 5.4.

These rules simulate industry-level semantics and ensure that runtime instances conform to domain-specific logic even in collaborative sessions.

5.4 | Collaborative Modeling Environment

The collaborative environment begins with a secure and role-aware login interface that authenticates users and routes them to either the web-based textual editor or the desktop graphical/

```

26 @Check
27 public void checkNameStartsWithCapital(Execution_Time_Constraint etc) {
28     if(etc.equals(null)){else
29     {
30         if (!Character.isUpperCase(etc.getName().charAt(0))) {
31             warning("Name should start with a capital",
32                 PkgDslPackage.Literals.EXECUTION_TIME_CONSTRAINT_NAME,
33                 INVALID_NAME);
34         }
35     }
36 }
37

```

FIGURE 7 | Validation rule specified on the automotive DSML.

tree editor, depending on assigned responsibilities. This design underpins the blended modeling paradigm, enabling stakeholders to contribute through both textual (EATEXT) and structural (EAXML) representations. Role-sensitive redirection is managed by a custom URI scheme protocol, which allows browser-initiated transitions to the desktop environment with embedded session details.

```

eatop://launch?user=james&model=volvo
\_dsml&session=20250723\_1732

```

Web users access an Xtext-powered editor (Figure 4) supporting syntax highlighting, validation, auto-completion, and log tracking, while desktop users work with structured visual representations. The workspace consists of two synchronized panels: a role-specific DSML editor and a read-only merger view. The merger panel aggregates all contributions in real time, annotated with contributor identity and time stamps for traceability (Figure 8): //The above part is Updated by: martin on 2025-07-08 15:35:01

This focuses on provenance, role-awareness, and traceability. Implemented rules:

- MR-5: Provenance rule: Every contribution receives identity + time stamp metadata.
- MR-1: Reference model rule: The merge panel compares all contributions against the reference instance.

- MR-2: Role-scope rule: Users only see and commit changes within their role-defined scope.

The interface also supports on-the-fly validation based on custom rules defined within the DSML grammar. As illustrated in Figure 8, the activation of validation rule 2 (which enforces naming conventions) results in a warning when a constraint name begins with a lowercase letter. A yellow warning triangle appears alongside an informative message, helping the user to adhere to defined modeling standards without disrupting the editing workflow. This is a nonblocking rule that provides stylistic feedback.

Implemented rules:

- MR-3: Validity rule: The model remains valid but issues a warning (soft validation).
- MR-5: Provenance rule: The system logs who introduced the naming violation.
- MR-6: Consistency rule: Even with warnings, the merged model is still kept consistent across all views.

Further, validation rule 1, which ensures uniqueness among START elements in *Execution_Time_Constraint* constructs, comes into effect during model merging. As shown in Figure 8, if two users inadvertently assign the same START value, the merged model triggers a red-underline error accompanied by a descriptive message:

"Execution_Time_Constraint START has to be unique"

This demonstrates the behavior of the framework's built-in validation engine during a merge operation. In this example, two *Execution_Time_Constraint* elements are introduced with identical start-constraint identifiers, which violates the rule that execution start points must remain unique within a timing specification. The merge interface highlights this conflict by marking the duplicated nodes and displaying an inline diagnostic message. As shown in Figure 8a, the system prevents automatic integration of such inconsistencies and prompts the reviewer to

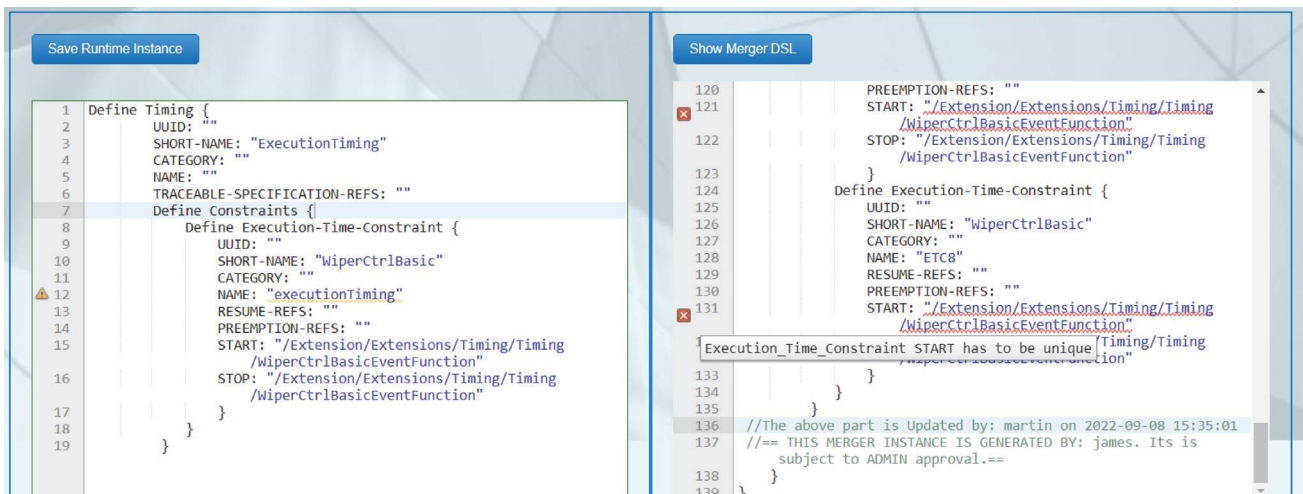


FIGURE 8 | Model merger and validation errors.

resolve the naming collision before the merged instance can be committed. This example illustrates how the framework enforces domain-specific constraints and ensures that timing models remain structurally consistent during collaborative editing. This example shows a semantic conflict and the system prevents merging due to a domain-specific violation.

Implemented rules:

- MR-3: Validity rule: The contribution fails semantic validation ($V(C_i) = \text{false}$) because two START values are identical.
- MR-4: Conflict resolution rule: The system detects a conflict between user contributions targeting the same constraint.
- MR-6: Consistency rule: The merge is blocked until the conflict is resolved, ensuring a consistent final model.
- MR-5: Provenance rule (implicit): The diagnostic messages include contributor time stamps and identities.

This runtime validation helps identify and resolve structural conflicts early in the development process, ensuring the integrity of the collective model. Thus, the collaborative environment enabled to observe representative merge behaviors that illustrate the practical functioning of the MR-1–MR-7 rules. During multiuser editing sessions, several conflict scenarios naturally emerged, most notably duplicated *Execution_Time_Constraint*.START values, inconsistent naming introduced by distributed edits, and concurrent structural updates targeting the same model element. Across the study, six such conflicts were detected and handled through the framework’s merge protocol: syntactic and naming conflicts were automatically resolved through precedence and validation checks, whereas semantically critical violations such as duplicate START assignments were escalated for manual reviewer approval. Each conflict instance was logged with contributor metadata and resolution status, forming an auditable trace that reinforces the transparency of the collaborative workflow. While these observations do not constitute a full empirical evaluation, they provide concrete evidence that the merge rules operate correctly in practice and support reliable and real-time collaboration within the DSML environment.

To ensure reliability, the environment incorporates a dual-persistence strategy. An *auto-save mechanism* captures model states every 10 s, while users can also manually trigger persistent saves or reload their last stored instance. This guarantees resilience against power loss or browser crashes and supports transparent recovery. All edits are logged with contributor details and time stamps, forming an immutable audit trail accessible to administrators for compliance and oversight.

Overall, this integrated setup balances usability, accountability, and semantic consistency. By combining role-sensitive access, bidirectional synchronization, validation-driven editing, and persistent logging, the environment enables distributed teams to collaboratively design and refine DSML models with fault tolerance, transparency, and domain-level rigor.

Within the desktop environment, users benefit from the simultaneous availability of textual exactness and graphical intuition, enabling tasks such as structural refinement, inspection of functional hierarchies, and examination of architectural dependencies. Once edits are completed, the updated model

state is committed to the shared repository through the back end services, ensuring that the desktop session remains aligned with the global collaborative workspace. This demonstrates that the proposed framework can sustain a blended modeling experience across modalities, thereby validating the practical feasibility of the architectural concepts introduced earlier. When users edit offline and later reconnect, the following rules apply:

Implemented rules:

- MR-7: Offline reintegration rule: Offline edits restored to user session only.
- MR-2: Role-scope rule: Offline edits are reviewed by higher-role users before merging.
- MR-4: Conflict resolution rule: Offline changes may conflict and require resolution.

5.5 | XAI Log Analysis and Model Traceability

A core innovation of the framework is the XAI Log Analysis Engine, designed to strengthen model traceability, accountability, and transparency in collaborative modeling. Whenever a user persists a runtime instance, the system automatically generates logs containing metadata such as username, time stamp, session ID, and the corresponding model snapshot.

Unlike conventional logs that demand manual inspection, the proposed engine integrates a LLM-based summarizer to translate raw logs into concise and human-readable narratives. These summaries not only describe what changes occurred but also contextualize them in terms of purpose, impact, and author. Figure 9 illustrates the AI-generated model activity summaries for a set of recent user actions. For instance, a log summary such as:

"An AI assistant has been tasked with summarizing the collaborative modeling activity of a team of users. Changes made by user "martin" to the system model: Added: Name: "Misbah awan 2" Removed: Name: "Misbah awan"

Such outputs clearly capture the syntactic update, the responsible actor, and the time stamp, while also extending to domain-specific modifications such as identifier renaming or structural edits.



FIGURE 9 | Explainable AI log analysis.

Following AXI rules are demonstrated in Figure 9, which are as follows:

AXI Rule 1: Contribution traceability rule:

Directly implemented.

Each entry in the log includes:

The username ("martin," "admin"),

The timestamp,

The action type (added, removed, renamed),

The affected model element ("Name: Misbah awan 2," "ETC12345 → ETC1234").

This corresponds exactly to Rule 1, where every modification is recorded with contributor identity, temporal metadata, and model context.

AXI Rule 4 - Semantic role impact rule:

Demonstrated through "admin" actions.

The last log entry shows actions performed by admin, a higher-privilege role:

Administrator added the following fields...

This aligns with Rule 4, where edits by high-privilege users (e.g., architects, integrators, and administrators) are given higher interpretive weight and appear as "finalizing actions" in summaries.

AXI Rule 5 - Summary generation rule:

Fully demonstrated.

Each block in Figure 9 is an LLM-generated narrative summarizing multiple low-level operations. For example:

"As per the updated system model, "ETC12345" has been renamed to "ETC1234"..."

This corresponds to Rule 5, where the presence of multiple modifications triggers the generation of a coherent team-level summary.

These AI-driven narratives serve dual purposes: (i) they reinforce model governance by making design evolution interpretable for both technical and managerial stakeholders and (ii) they promote a culture of collaborative transparency, where contributions are visible, auditable, and accountable. The logs may also be archived or queried for audits, design reviews, or managerial oversight, making them particularly valuable in large-scale automotive and other regulated system development environments.

To contextualize the advantages of XAI-based summaries, we contrast them with traditional textual logs that preserve complete EAXML model snapshots, demonstrating how the XAI mechanism converts low-level structural updates into concise and human-readable interpretations. While the conventional logs preserve complete structural detail, they are often verbose and difficult to interpret, requiring users to manually scan large serialized fragments to identify what has changed. By contrast, the XAI-generated narratives distill these updates into compact and human-readable statements that explicitly highlight added,

removed, or modified elements together with the responsible user and time stamp as shown in Figure 9. This abstraction enables engineers to rapidly understand the intent and impact of collaborative edits without navigating low-level syntax. Importantly, the XAI layer does not replace or alter the authoritative logs; rather, it acts as an interpretive layer that enhances traceability, accelerates review cycles, and improves situational awareness during collaborative modeling. This comparison clarifies the complementary role of the XAI summaries and motivates future empirical evaluation of their accuracy and perceived usefulness.

In conclusion, the framework enables real-time and multiuser collaboration across textual and graphical/tree models, integrating blended interfaces, auto-backup, model merging, and AI-driven logging to meet both functional and nonfunctional demands of embedded system design. Validated on the BUMBLE ITEA Volvo case study, it demonstrates industrial relevance and technical feasibility while advancing research in collaborative modeling, MDE, and explainable model evolution, establishing a robust and extensible foundation for human-centric and collaborative design.

6 | Performance Evaluation

Since the framework is intended to support multistakeholder modeling workflows commonly used in safety-regulated automotive embedded systems, its ability to maintain responsiveness and reliability under concurrent load was rigorously assessed. The evaluation focused on scalability, latency, and error resilience across two synchronized environments: a web-based textual editor and a desktop-based tree/graphical editor, linked through a bidirectional synchronization layer. While the desktop client reflects local engineering deployments, the web interface serves globally distributed users, making wide-area network performance a critical factor.

6.1 | Experimental Setup

The test workflow replicated a full modeling cycle, including user login, retrieval of the latest runtime instance, edits with persistence, merged view generation, and session termination, executed simultaneously across web and desktop clients on a shared back end. To emulate realistic Internet conditions, the web server was exposed via Cloudflare tunneling, introducing genuine latency and routing variability.

6.2 | Methodology and Tools

To ensure robust and reproducible measurements, three widely recognized performance-testing platforms were employed:

1. BlazeMeter [21]—A JMeter-based cloud service used to execute the captured *.jmx* scenario with up to 50 concurrent users in its free tier.
2. LoadView [22]—A geographically distributed load-testing service providing globally injected virtual users to measure real-world network impact.
3. Apache JMeter [23]—A desktop-based, open-source load testing tool, enabling stress tests with up to 100 simulated users.

TABLE 1 | Comparative results from performance testing tools.

Platform	Max users	Duration	Avg. Resp. time (s)	Avg. BW (MB/s)	Avg. throughput (Hits/s)	Error (%)	Load injectors	Latency (ms)
BlazeMeter	50	16 min 33 s	2.46	1.02	16.71	0.0	—	20
LoadView	09	12 min 36 s	27.16	—	24.0	0.0	2	30
JMeter	100	20 min 00 s	19.00	—	3.0	0.0	—	15

Using multiple tools enabled cross-validation of key operations such as model retrieval, delta synchronization, and real-time merging. Table 1 presents the consolidated results.

6.3 | System Responsiveness

The system demonstrated consistent stability across all platforms (Table 1), successfully sustaining 100 concurrent users without errors. Average response time increased predictably with load, from 2.46 s at 50 users (BlazeMeter) to 19 s at 100 users (JMeter). Latency remained low, between 15 and 30 ms, indicating that back end event dispatching, validation, and persistence subsystems did not become bottlenecks under stress. Importantly, synchronization between the web-based editor and the desktop editor remained seamless, with no observed desynchronization or missed updates, even at peak loads.

These findings confirm that the framework's event-driven architecture, transaction-safe persistence layer, and delta-based synchronization protocol collectively support high concurrency while maintaining real-time responsiveness and semantic integrity. This level of performance meets the practical demands of collaborative modeling in distributed MDE contexts, providing confidence in the system's operational viability for large-scale deployment.

7 | Discussion

This work directly addresses the two research questions posed in Section 1 by demonstrating how real-time collaborative blended modeling can be realized with semantic rigor, controlled conflict resolution, and explainable traceability. With respect to RQ1, the proposed framework shows that synchronized textual (EAText) and graphical/tree-based (EAXML) modeling can be maintained through a shared abstract syntax, event-driven synchronization, and a structured merge protocol. By decoupling representation, persistence, and synchronization layers, the framework preserves bidirectional consistency while remaining extensible. Fine-grained RBAC ensures role-aware collaboration, deterministic checkpointing ($T_c = 10$ s) supports fault tolerance aligned with ISO 26262 principles, and conflict resolution combines syntactic validation, semantic checks, and precedence policies with full provenance annotation.

With respect to RQ2, the XAI log analysis engine demonstrates how explainability can be integrated into collaborative blended modeling environments to enhance traceability, interpretability, and awareness of model evolution, while preserving determinism, privacy, and modeling correctness. The XAI layer operates on validated collaboration events and synchronized model updates, translating low-level EAXML edits and merge actions

into domain-aware natural language summaries. This enables stakeholders to understand how and why the model is evolving, without manually inspecting raw logs or version histories. Traceability is ensured through the association of user roles, time stamps, and semantic change descriptions in the generated summaries, supporting accountability and design audits. Awareness of model evolution is strengthened by continuous and human-readable narratives that expose the progression of design decisions and conflict resolutions across distributed teams. Importantly, the explainability engine is architecturally decoupled from the core synchronization and transformation pipeline, ensuring that determinism and modeling correctness are not compromised. It does not modify model states or merge logic, but only interprets already validated events. Privacy is preserved through role-aware filtering aligned with the RBAC policies of the collaboration engine, ensuring that users only receive summaries for authorized model segments. Stress testing further confirms that explainable collaboration can coexist with real-time performance, achieving scalability up to 100 concurrent users with stable latency (15–30 ms) and no missed updates. To the best of our knowledge, this is the first integration of XAI for activity summarization and evolution awareness in synchronized textual and graphical blended modeling environments.

An additional strength of the proposed framework is its public availability as an open-source platform [20], which significantly enhances its practical relevance and research impact. By releasing the full implementation, including the collaboration engine, synchronization services, and XAI log analysis modules, the framework is positioned as a reusable and extensible infrastructure for both academic experimentation and industrial replication. For researchers, this enables independent validation, comparative benchmarking, and extension toward new modeling languages or collaboration scenarios. For practitioners, open availability lowers adoption barriers, facilitates pilot deployment, and supports technology transfer into existing engineering toolchains. This openness directly supports the long-term objective of large-scale empirical evaluation and replication studies, which are outlined as a future research direction. In contrast to many MDE prototypes that remain closed or partially documented, the public release of this framework enables transparent scrutiny, reproducibility, and community-driven evolution.

7.1 | Novelty and Positioning

While individual aspects of the proposed framework (e.g., synchronization, versioning, or AI-based analysis) have been explored in prior studies of other domains, their integration within a single, real-time, and role-aware collaborative blended modeling environment constitutes the primary novelty of this work. Existing approaches either focus on syntactic synchronization without

TABLE 2 | Positioning of the proposed framework against closely related collaborative and blended modeling approaches.

Aspect	Van Tendeloo and Vangheluwe [10]	Exelmans et al. [11]	Proposed framework (this work)
Modeling paradigm	Blended modeling with separated abstract and concrete syntax	Blended modeling with optimistic versioning and semantic merge	AI-augmented collaborative blended modeling with unified abstract syntax
Notation integration	Textual + graphical (DSL-specific)	Textual + tree-based/graphical views	Textual (EAText) + tree/graphical (EAXML) + extensible DSMLs
Synchronization strategy	Conceptual abstract and concrete syntax separation; no operational protocol defined	Semantic merge operations, not real-time	Real-time, event-driven bidirectional synchronization across views
Collaboration model	Not explicitly defined	Collaborative editing, tool-centric workflows	Role-aware collaboration with protocol-level redirection and RBAC
Conflict handling	Not addressed	Semantic merge at model level	Multilayered syntactic + semantic conflict resolution with traceability
Versioning and persistence	Not detailed	Limited, no fault tolerance guarantees	Time-triggered checkpointing and fault-tolerant persistence
Explainability/traceability	Not supported	Not supported	AI-based explainable activity summarization and change rationale
Tool support	Partial	Partial	Full (Web–desktop integrated platform)
Industrial validation	Not reported	Limited experimental scenarios	Validated on industrial automotive (Volvo wiper timing) case study
Open source	No	No (provided link not accessible)	Yes

collaboration semantics [10], provide conflict-tolerant versioning without persistent storage or intelligent assistance [11], or support runtime synchronization across limited notations without role awareness or explainability [11]. To further clarify the architectural and operational novelty of the proposed framework, Table 2 positions it against the most closely related collaborative and blended modeling approaches identified in the literature. While existing works address individual aspects such as versioning or semantic merging, none integrate real-time multiview synchronization, role-aware collaboration, fault-tolerant persistence, and AI-driven explainability within a single cohesive architecture. In contrast, the framework presented in this research uniquely combines real-time multiview synchronization, fine-grained RBAC, semantic conflict resolution, persistent versioning, and XAI-driven activity summarization in a unified architecture. To the best of our knowledge, no prior MDE or blended modeling platform embeds XAI directly into the collaborative modeling pipeline to enhance transparency, traceability, and governance. This integrated, human-centered perspective advances collaborative blended modeling beyond tool interoperability toward intelligent and explainable engineering environments.

7.2 | Threats to Validity

While the proposed framework demonstrates robustness and scalability, several validity concerns remain. Internal validity is limited by controlled testing environments that may underestimate variability in real-world networks and by conflict resolution strategies

evaluated only on predefined scenarios. External validity is constrained by reliance on the Volvo car wiper timing package and participants already familiar with MDE, raising questions about applicability across domains such as aerospace or medical systems and about transferability to practitioners with different tooling backgrounds and industrial workflows. Construct validity is partial since metrics like latency and throughput capture scalability, but not user satisfaction, cognitive load, or the semantic accuracy of the XAI log engine, which was only assessed informally and not through structured human–subject evaluation or practitioner-based usability studies. Reliability concerns arise from slight variations across BlazeMeter, LoadView, and JMeter due to tool-specific methodologies. To mitigate these threats, future work will involve geographically distributed trials, diverse participant studies, formal semantic benchmarks for AI summarization, and standardized performance protocols to ensure reproducibility and generalizability.

7.3 | Future Work

Several avenues for continued research extend naturally from this work. First, a comprehensive empirical evaluation is planned to assess the framework's effectiveness beyond architectural and infrastructural validation. This includes controlled user studies and comparative experiments designed to evaluate usability, collaboration efficiency, merge quality, reduction of modeling inconsistencies, and the perceived usefulness of XAI summaries. Such human-in-the-loop assessments represent a distinct

empirical phase and will complement the functional and performance-oriented validation presented in this paper.

Second, the XAI-based log analysis engine will be enhanced with more expressive natural language reasoning, refined prompt-engineering strategies, and domain-aware summarization aligned with EAST-ADL semantics. Future enhancements will incorporate dynamic model selection, reliability scoring, and robustness checks to further reduce hallucination risks and improve interpretability.

Finally, the framework will be subjected to broader industrial replication studies beyond the Volvo wiper case study. Planned pilot deployments with additional automotive partners will investigate long-term usability, organizational adoption, interoperability with existing toolchains, and scalability across multiteam and multimodel environments. These industrial replication efforts will provide deeper insight into real-world applicability and constitute a necessary step toward comprehensive validation in production-grade engineering contexts.

8 | Conclusion

This research has presented a rigorously engineered AI-augmented collaborative framework to support the development of automotive embedded systems, built around the principles of blended modeling to maintain semantic coherence across heterogeneous notations. By enabling synchronized, bidirectional transformations between textual representations and tree-based/graphical models, the framework ensures seamless multiview consistency, allowing domain experts and software engineers to work in their preferred notations without loss of meaning or fidelity. Its architecture integrates RBAC, explainable log analytics powered by LLMs, fault-tolerant persistence strategies, and real-time semantic conflict resolution, collectively delivering reliability, transparency, and traceability in distributed multiuser workflows. Validation using an industrial-scale automotive case study has demonstrated the framework's operational resilience, scalability, and theoretical soundness in collaborative modeling scenarios representative of real-world development environments. While the evaluation was conducted in the automotive domain, the modular, domain-agnostic architecture, extensible AI analytics pipeline, and blended modeling infrastructure position the framework as a reusable solution for other high-assurance embedded systems, including aerospace, medical devices, and industrial automation.

Acknowledgments

This project receives partial funding from the Higher Education Commission of Pakistan via the NRPU MRED initiative under Grant 20-15651, as well as partial support from the ITEA3 BUMBLE Project (Grant 18006).

Funding

This study was funded by the Higher Education Commission, Pakistan (Grant 20-15651) and the ITEA3 (Grant 18006).

Conflicts of Interest

The authors declare no conflicts of interest.

Data Availability Statement

The data that support the findings of this study are openly available in *CollaborativeBlendedModelingFramework* at Awan [20]. However, the EATOP module that supports the findings of this study is available from Volvo Group, Sweden. Restrictions apply to the availability of EATOP, which was used under license for this study.

References

1. M. Franzago, D. Di Ruscio, I. Malavolta, and H. Muccini, "Collaborative Model-Driven Software Engineering: A Classification Framework and a Research Map," *IEEE Transactions on Software Engineering* 44, no. 12 (2018): 1146–1175.
2. M. Maróti, T. Kecskés, R. Kereskényi, et al., "Next Generation (meta) Modeling: Web- and Cloud-Based Collaborative Tool Infrastructure," *MPM@MoDELS* 1237 (2014): 41–60.
3. Eclipse Foundation, "Eclipse Modeling Framework (emf)," 2023, <https://www.eclipse.org/modeling/emf/>.
4. J. Corley, E. Syriani, and H. Ergin, "Evaluating the Cloud Architecture of Atompm," in *Proceedings of the 4th International Conference on Model-Driven Engineering and Software Development*, (SciTePress, 2016): 339–346.
5. C. Debreceni, G. Bergmann, M. Búr, I. Ráth, and D. Varró, "The Mondo Collaboration Framework: Secure Collaborative Modeling Over Existing Version Control Systems," in *Proceedings of the 11th Joint Meeting on Foundations of Software Engineering*, (Association for Computing Machinery, 2017): 984–988.
6. J. L. Cánovas Izquierdo and J. Cabot, "Collaboro: A Collaborative (meta) Modeling Tool," *PeerJ Computer Science* 2 (2016): e84.
7. S. N. Voogd, K. Aslam, L. Van Gool, B. Theelen, and I. Malavolta, "Real-Time Collaborative Modeling Across Language Workbenches—a Case on JetBrains Mps and Eclipse Spoofox," in *MODELS-C. ACM/IEEE*, (2021): 16–26.
8. D. Wüest, N. Seyff, and M. Glinz, "Flexisketch: A Lightweight Sketching and Metamodeling Approach for End-Users," *Software & Systems Modeling* 18, no. 2 (2019): 1513–1541.
9. M. M. Awan, M. W. Anwar, W. H. Butt, and F. Azam, "A Blended Modeling Framework for Real-Time Design and Verification of Safety-Critical Embedded Systems," *PLOS ONE* 20, no. 12 (2025): e0337604.
10. Y. Van Tendeloo and H. Vangheluwe, "Unifying Model- and Screen Sharing," in *IEEE 27th International Conference on Enabling Technologies: Infrastructure for Collaborative Enterprises (WETICE)*, (IEEE, 2018): 127–132.
11. J. Exelmans, J. Pietron, A. Raschke, H. Vangheluwe, and M. Tichy, "A New Versioning Approach for Collaboration in Blended Modeling," *Journal of Computer Languages* 76 (2023): 101221.
12. J. Pietron, A. Raschke, J. Exelmans, and M. Tichy, "Collaboration and Versioning Framework-A Systematic Top-down Approach," in *2023 ACM/IEEE International Conference on Model-Driven Engineering Languages and Systems Companion (MODELS-C)*, (IEEE, 2023): 767–777.
13. I. David, M. Latifaj, J. Pietron, et al., "Blended Modeling in Commercial and Open-Source Model-Driven Software Engineering Tools: A Systematic Study," *Software and Systems Modeling* 22 (2022).
14. O. B. Badreddin, T. C. Lethbridge, and A. Forward, "A Novel Approach to Versioning and Merging Model and Code Uniformly," in *Proceedings of the 2nd International Conference on Model-Driven Engineering and Software Development (MODELSWARD)*, (IEEE, 2014): 254–263.
15. Apache, "Apache Subversion," 2025, <https://subversion.apache.org/>.
16. M. Latifaj, H. Taha, F. Ciccozzi, and A. Cicchetti, "Cross-Platform Blended Modelling With JetBrains mps and Eclipse Modeling Framework," in *19th International Conference on Information Technology-New Generations (ITNG. 2022)*, (Springer, 2022): 3–10.

17. L. Addazi and F. Ciccozzi, “Blended Graphical and Textual Modelling for UML Profiles: A Proof-of-Concept Implementation and Experiment,” *Journal of Systems and Software* 175 (2021): 110912.
18. M. W. Anwar, F. Ciccozzi, and A. Bucaioni, “Enabling Blended Modelling of Timing and Variability in EAST-ADL,” in *Proceedings of SLE*, (Association for Computing Machinery, 2023): 169–180.
19. R. Palin, D. Ward, I. Habli, and R. Rivett, “ISO 26262 Safety Cases: Compliance and assurance,” in *6th IET International Conference on System Safety*, (IET, 2011).
20. M. Awan, “CollaborativeBlendedModelingEnvironment Tool,” 2026, Accessed: Jan. <https://github.com/MisbahAwan/CollaborativeBlendedModelingFramework>.
21. P. Memon, T. Hafiz, S. Bhatti, and S. S. Qureshi, “Comparative Study of Testing Tools Blazemeter and Apache Jmeter,” *Sukkur IBA Journal of Computing and Mathematical Sciences* 2, no. 1 (2018): 70–76.
22. R. Rizal and A. R. Ruuhwan, “Gtmetrix, Webaim Dan Loadview,” (2020).
23. D. Nevedrov, “Using Jmeter to Performance Test Web Services,” (2006): 1–11, Published on Dev2Dev (2006): 1–11.

Appendix A

Table A1 provides a consolidated list of acronyms and abbreviations used throughout the manuscript, along with their full forms, to support clarity and ease of reference for readers.

TABLE A1 | List of acronyms and descriptions.

Acronym	Description
EAST-ADL	Electronics architecture and software technology-architecture description language
DSML	Domain-specific modeling language
XAI	Explainable artificial intelligence
LLMs	Large language models
EAText	EAST-ADL textual representation
EAXML	EAST-ADL XML representation
EMF	Eclipse modeling framework
EATOP	EAST-ADL tool platform
RBAC	Role-based access control
DAO	Data access objects
NLP	Natural language processing
HTTP	Hypertext transfer protocol
JSON	JavaScript object notation
TCP/IP	Transmission control protocol/internet protocol
MVC	Model-view-controller
AJAX	Asynchronous JavaScript and XML
RESTful API	Representational state transfer application programming interface
LLaMA	Large language model meta AI
BUMBLE	Blended modeling for enhanced software and systems engineering
ITEA	International Test and Evaluation Association